

Why Your Model Fails on Hard Problems

The claim

A model that learns from demonstrations can fail on hard problems. The reason is not a lack of ability. The demonstrations did not cover that difficulty level. Add one demonstration at the harder level, and the model recovers.

We built an interactive demo that lets you see this happen in real time.

What we built

Our project is an entry for DataForge 2026, Pathway track. We trained a small transformer (~4.2 million parameters, decoder-only, 6 layers) on a sorting task from the ARC family. The task: given colored bars of varying heights on a 10x10 grid, sort them left to right by height.

The complexity parameter is the number of bars, ranging from 2 to 8. We trained the model on all complexities with “covered” demonstrations. Covered means that at least one of the three example pairs shown to the model matches the query’s difficulty. The model learned to solve every level.

At inference time, we do not retrain. We do not change the model weights. We change only the demonstrations. In the “uncovered” condition, all three demos show tasks with only 2 or 3 bars. The query still asks the model to sort 8 bars. The model sees the same question. It has the same weights. It fails.

What we found

The results were sharp. At complexity 8, the model scored 100% exact match with covered demonstrations. With uncovered demonstrations, it scored 0%. The gap is 100 percentage points. The model can sort 8 bars. It proved that during training and during covered inference. It cannot figure out how to sort 8 bars by looking at examples that only sort 2 or 3.

This is not a bias-variance problem. The model capacity did not change. The training data did not change. The only variable is which examples the model reads as input context at inference time. The phenomenon is about in-context learning. The model infers the task rule from the demonstrations it sees. When the demonstrations do not cover the difficulty it faces, that inference breaks.

Pathway’s BDH-CQ system (150 million parameters) shows the same cliff on the same task family. At ordering length 8, BDH-CQ scores 0 out of 24 with short context and 12 out of 24 with supported context (arXiv:2608.09888, Table 3). The effect scales from our 4.2M toy model to a production-grade system.

The BDH connection

Our demo includes a side-by-side comparison of two memory architectures. On the left: a standard Transformer KV cache. It grows with each token. Attention re-reads every slot on each step. On the right: a Hebbian synaptic matrix, a simplified illustration of the memory substrate from Pathway’s Dragon Hatchling architecture (arXiv:2509.26507).

The Hebbian memory uses a fixed-size matrix. Each demonstration writes into it with an outer-product update. Reading multiplies the query by the accumulated matrix. The matrix does not grow. But it has finite capacity. When the matrix fills up, older writes interfere.

This is a simplified version. It omits low-rank compression, the positional operator, excitatory and inhibitory circuits, and gating. We label it as such in the demo. The point is not to replicate BDH. The point is to show the learner how two architectures store demonstrations differently, and why coverage matters to both.

The cost difference is large. BDH-CQ processes one task for \$0.00070. HRM, which adapts by running a backward pass on augmented demo pairs, costs \$1.48 per task (arXiv:2506.21734). TRM, which learns an identity embedding per puzzle, costs \$1.76. Recurrent state absorption is orders of magnitude cheaper than optimization-based adaptation.

How the demo works

The interactive artifact opens with a guided walkthrough that takes under 90 seconds. The learner starts at complexity 3 with covered demonstrations. The model sorts 3 bars correctly. The learner drags the complexity slider to 8. The model still works because the demonstrations cover that difficulty. The learner then flips a toggle to remove the matching demonstration. The model fails. Nothing about the model or the question changed. Only the examples did.

After the guided flow, all controls unlock. The learner can explore every complexity and toggle between covered and uncovered conditions freely.

The frontend loads predictions from a precomputed JSON file. This means the demo works instantly, with no loading delay. If the browser loaded the ONNX model for live inference, a visible badge in the top right corner reads LIVE. Otherwise it reads PRECOMPUTED. We never hide this. If the output comes from a file, the badge says so.

We took this design approach from Transformer Explainer (Georgia Tech, CHI 2026). That project runs GPT-2 live in the browser via ONNX Runtime Web with progressive disclosure.

The tech stack is React, Vite, and Tailwind for the frontend. PyTorch for training. ONNX Runtime Web for optional live inference.

What we learned

Three lessons stood out.

First: the effect is sharper than we expected. We planned for a 30 percentage point gap at complexity 8. We got 100. The model either gets the demos it needs, or it produces nonsense. There is no graceful degradation.

Second: honest labeling matters. Latent reasoning models like Coconut (arXiv:2412.06769) process demonstrations internally without visible chain-of-thought tokens. A demo that hides whether its output is live or precomputed undermines the trust the learner needs. The badge is small, but it is always visible.

Third: one controlled variable teaches more than ten. Our demo has two controls: a complexity slider and a coverage toggle. That is enough to let the learner discover, test, and confirm the claim on their own.

The source code is public. Every number in the evidence table traces to a specific section of a specific paper. We did not run BDH-CQ. We report the numbers its authors published.